

SISTEMAS INTELIGENTES DISTRIBUIDOS

Q2 2025–2026

Práctica 2

Análisis de algoritmos de aprendizaje por refuerzo sobre
FROZENLAKE-V1

Autores

Martí Checa
Óscar Nagl
Andreu Puerto

11 de mayo de 2026

Contents

1	Introducción	3
2	Descripción del entorno FrozenLake	3
2.1	Características del entorno	3
2.2	Retos del entorno	4
3	Hipótesis y parámetros de experimentación	4
3.1	Hipótesis	4
3.2	Parámetros y rangos de experimentación	5
3.3	Métricas de evaluación	5
4	Estructura del código y decisiones de diseño	6
4.1	Módulo de configuración (<code>config.py</code>)	6
4.2	Infraestructura de pruebas y entorno (<code>main.py</code>)	6
4.3	Motor de experimentación científica (<code>experiments.py</code>)	6
5	Iteración de Valor	6
5.1	Fundamentos teóricos	6
5.2	Implementación	7
5.3	Resultados experimentales	7
5.4	Análisis del efecto de los parámetros	8
5.5	Fortalezas y debilidades	8
6	Estimación Directa	9
6.1	Fundamentos teóricos	9
6.2	Implementación	10
6.3	Resultados experimentales	10
6.4	Análisis del efecto de los parámetros	11
6.5	Fortalezas y debilidades	11
7	Q-Learning	12
7.1	Fundamentos teóricos	12
7.2	Implementación	12
7.3	Resultados experimentales	12
7.4	Análisis del efecto de los parámetros	16
7.5	Fortalezas y debilidades	16
8	REINFORCE	17
8.1	Fundamentos teóricos	17
8.2	Implementación	17
8.3	Resultados experimentales	17
8.4	Análisis del efecto de los parámetros	19
8.5	Fortalezas y debilidades	19
9	Análisis de escalabilidad	20
10	Comparación global y conclusiones	21
10.1	Verificación de las hipótesis	21
10.2	Tabla resumen del experimento principal	22
10.3	Conclusiones finales	22

10.4 Recomendaciones prácticas	23
Referencias	24

1 Introducción

En esta práctica se ha realizado un análisis exhaustivo y comparativo del rendimiento de diferentes algoritmos de Aprendizaje por Refuerzo (RL) aplicados al entorno **FrozenLake-v1** de Gymnasium. A diferencia de aproximaciones más sencillas trabajadas en la sesión introductoria de laboratorio, este estudio se centra estrictamente en la resolución del Proceso de Decisión de Markov (MDP) subyacente operando en su *modo difícil*, caracterizado por una alta estocasticidad de las transiciones (`is_slippery=True`) y, opcionalmente, por mapas de mayor tamaño generados procedimentalmente.

Para llevar a cabo este análisis se ha implementado una batería de agentes que cubren los principales paradigmas del aprendizaje por refuerzo:

- **Basados en modelo:** Iteración de Valor (con acceso completo a $P(s'|s, a)$) y Estimación Directa, que estima el modelo a partir de la experiencia.
- **Basados en valor y experiencia (model-free):** Q-Learning, que aproxima directamente la función acción-valor mediante diferencias temporales.
- **Basado en gradiente de política:** REINFORCE, implementado como extensión para aportar una perspectiva adicional sobre la optimización directa de políticas.

El estudio se apoya en una arquitectura de software modular y orientada a la experimentación científica reproducible. El sistema consta de un núcleo de configuración centralizado (`config.py`), un gestor de entornos que permite la creación de mapas personalizados y la aplicación de esquemas de recompensa o *reward shaping* (`main.py`), y un motor de experimentación masiva (`experiments.py`). Este último está diseñado para recolectar métricas avanzadas (tasa de éxito en evaluación, recompensa media, tiempos de convergencia y similitud con políticas oráculo) sometiendo a los agentes a diferentes tamaños de mapa (desde las cuadrículas estándar de 4×4 y 8×8 hasta dimensiones superiores de 10×10 y 12×12) y a distintos grados de incertidumbre modulando la probabilidad de éxito de las acciones (`success_rate`).

El presente documento recoge los resultados obtenidos, el análisis comparativo y las conclusiones del estudio, contextualizando cada observación con las propiedades teóricas vistas en las sesiones magistrales y de laboratorio.

2 Descripción del entorno FrozenLake

2.1 Características del entorno

El entorno **FrozenLake-v1** simula la navegación de un agente sobre la superficie de un lago parcialmente congelado. El objetivo es trazar una ruta desde la casilla de inicio (S) hasta la meta (G), evitando caer en una serie de agujeros de hielo (H) distribuidos por el mapa. Sus características definitorias son:

- **Espacio de observaciones (estados):** espacio discreto representado por una cuadrícula. En este estudio se evalúa la escalabilidad partiendo de mapas base de 4×4 (16 estados) y 8×8 (64 estados), y se extiende mediante generación procedimental a resoluciones de 10×10 (100 estados) y 12×12 (144 estados).
- **Espacio de acciones:** el agente dispone de cuatro movimientos discretos en cada estado (izquierda, abajo, derecha y arriba).
- **Dinámica de transición:** el entorno opera con `is_slippery=True`. Las transiciones no son deterministas: cuando el agente ejecuta una acción direccional, existe una probabilidad (modificable a través del parámetro `success_rate`) de que se mueva efectivamente en

la dirección deseada, y una probabilidad complementaria, repartida equitativamente, de resbalar perpendicularmente debido al hielo.

- **Señal de recompensa:** la formulación original proporciona un *feedback* extremadamente disperso (*sparse reward*), otorgando 0 puntos por cualquier transición y +1 punto únicamente al alcanzar la meta de forma exitosa. Para profundizar en el análisis se ha implementado un sistema de esquemas de recompensa (`reward_schedule`) que permite evaluar funciones alternativas, tales como pequeñas penalizaciones por paso o penalizaciones más severas por caer en los agujeros.
- **Estado inicial y objetivo:** el agente parte siempre de la celda superior izquierda (S) y debe alcanzar la celda inferior derecha (G).

2.2 Retos del entorno

La resolución de FrozenLake plantea desafíos técnicos significativos que pondrán a prueba las fortalezas y debilidades de cada algoritmo:

- **Alta estocasticidad y gestión del riesgo:** la combinación de superficies resbaladizas y estados terminales negativos (agujeros) obliga a los algoritmos a ir más allá de la simple búsqueda del camino más corto. Los agentes deben aprender a equilibrar el riesgo y la eficiencia, buscando trayectorias que maximicen la probabilidad de supervivencia antes que la longitud geométrica de la ruta.
- **Recompensa escasa y el problema del *credit assignment*:** al recibir retroalimentación útil solo al final del episodio, los métodos basados puramente en experiencia (Q-Learning o REINFORCE) se enfrentan a la dificultad de asignar correctamente el valor a las acciones tempranas que llevaron a la victoria. Esto exige estrategias de exploración robustas (ϵ -greedy) y tasas de aprendizaje muy bien calibradas.
- **Explosión del espacio de estados:** al escalar el mapa a configuraciones de 10×10 o 12×12 , la matriz de transiciones y el horizonte temporal requerido para alcanzar la meta crecen drásticamente. Esto supone una prueba de estrés directa para la eficiencia de los algoritmos: los métodos de planificación como Iteración de Valor sufrirán aumentos polinómicos en los tiempos de cómputo por iteración, mientras que los agentes *model-free* necesitarán incrementar masivamente la recolección de muestras y ajustar mucho más finamente su exploración para lograr la convergencia.

3 Hipótesis y parámetros de experimentación

3.1 Hipótesis

Antes de realizar los experimentos se plantearon las siguientes hipótesis, derivadas de la teoría vista en clase y de los resultados obtenidos en el laboratorio introductorio:

1. Los algoritmos basados en modelo (Iteración de Valor y Estimación Directa) convergerán más rápidamente que los basados en experiencia (Q-Learning y REINFORCE) cuando el modelo subyacente sea accesible o se pueda muestrear con suficiente densidad.
2. El factor de descuento γ tendrá un impacto significativo en el comportamiento del agente, favoreciendo políticas más conservadoras y robustas al ruido cuando $\gamma \rightarrow 1$.
3. Q-Learning mostrará una sensibilidad alta a hiperparámetros como la tasa de aprendizaje y el coeficiente de exploración, y exhibirá comportamientos bimodales entre seeds cuando la exploración sea insuficiente.

4. REINFORCE requerirá un número de episodios sensiblemente mayor para converger y mostrará la mayor varianza inter-seed, debido a la elevada varianza intrínseca del estimador de gradiente Monte Carlo.
5. Al aumentar el tamaño del mapa, los métodos *model-free* colapsarán antes que los métodos basados en modelo, ya que la *sparse reward* amplifica el problema del *credit assignment* en horizontes largos.
6. Al disminuir `success_rate`, todos los algoritmos perderán rendimiento de forma monótonica; el método más resistente será Value Iteration, al no depender de la calidad del muestreo.

3.2 Parámetros y rangos de experimentación

Para todos los algoritmos se han evaluado los siguientes parámetros comunes:

- **Factor de descuento** $\gamma \in \{0.9, 0.95, 0.99\}$.
- **Tamaño del mapa:** 4×4 , 8×8 , 10×10 , 12×12 (los dos últimos generados proceduralmente).
- **Estocasticidad (success_rate):** $\{0.20, 0.33, 0.60, 0.80, 1.00\}$, donde 1.00 corresponde a un entorno determinista y 0.20 a un entorno cuya probabilidad de moverse en la dirección deseada es menor que la de resbalar.
- **Esquema de recompensa:** *default* (sólo +1 en la meta), *hole penalty* (penalización de -1 por caer en agujero) y *step penalty* (penalización de -0.01 por paso).

Adicionalmente, para cada algoritmo se han analizado los hiperparámetros propios:

- **Iteración de Valor:** umbral de convergencia $\theta = 10^{-6}$.
- **Estimación Directa:** número de transiciones muestreadas $\in \{1000, 5000, 20000, 100000\}$.
- **Q-Learning:** número de episodios $\in \{500, 1000, 2000, 5000, 10000, 20000\}$, tasa de aprendizaje $\alpha \in \{0.05, 0.1, 0.3, 0.5\}$, coeficiente de exploración $\epsilon_0 \in \{0.3, 0.5, 0.8, 1.0\}$ con decaimiento exponencial.
- **REINFORCE:** tasa de aprendizaje $\in \{10^{-3}, 10^{-2}, 5 \cdot 10^{-2}, 10^{-1}\}$.

3.3 Métricas de evaluación

Para evaluar la calidad de los entrenamientos y construir el análisis, se han empleado las siguientes métricas:

- **Tasa de éxito en evaluación:** porcentaje de episodios en los que el agente alcanza la meta tras el entrenamiento, evaluado siempre con la política voraz (*greedy*) sobre 1000 episodios.
- **Recompensa media y número medio de pasos** por episodio.
- **Tiempo de entrenamiento** y, en su caso, número de iteraciones internas hasta la convergencia.
- **Optimalidad de la política:** porcentaje de estados en los que la política aprendida coincide con la de Value Iteration (`eval_policy_agreement_vi`), usada como oráculo dado que VI converge demostrablemente a π^* .
- **Varianza inter-seed:** todas las medias se reportan junto con su desviación típica sobre 5 seeds (3 seeds en los mapas 10×10 y 12×12 por coste computacional).

4 Estructura del código y decisiones de diseño

Para facilitar la experimentación y asegurar la reproducibilidad de los resultados, se ha desarrollado una arquitectura de software modular. Esta separación de responsabilidades permite modificar algoritmos o configuraciones del entorno de forma independiente sin afectar al resto del sistema.

4.1 Módulo de configuración (`config.py`)

Este archivo actúa como el cerebro de parámetros del proyecto: centraliza todos los hiperparámetros organizados en bloques (entorno, *reward shaping* y algoritmos) para evitar los *magic numbers* dispersos en el código. Aquí se definen valores críticos que garantizan la consistencia de las pruebas, tales como el factor de descuento γ , las tasas de aprendizaje α , el coeficiente de exploración ϵ y los criterios de convergencia θ .

4.2 Infraestructura de pruebas y entorno (`main.py`)

Este fichero contiene la lógica fundamental para la creación y validación de los agentes. Implementa la *factory* de entornos mediante la función `create_env`, que soporta la generación de mapas de cualquier tamaño y la inyección de diferentes grados de estocasticidad a través del parámetro `success_rate`. Además, proporciona la función `evaluate_agent`, encargada de medir el porcentaje de éxito y la recompensa media de una política ya entrenada en modo de explotación. Sirve como banco de pruebas y depuración rápida antes de proceder a la experimentación masiva.

4.3 Motor de experimentación científica (`experiments.py`)

Es el núcleo encargado de la ejecución automatizada de las pruebas. Está diseñado para lanzar baterías de ejecuciones de forma sistemática, abarcando desde las calibraciones iniciales de hiperparámetros hasta los experimentos principales de escalabilidad y varianza inter-seed. Este módulo garantiza el rigor empírico permitiendo guardar los resultados numéricos estructurados en archivos CSV y generar automáticamente las gráficas de aprendizaje y las trazas necesarias para el análisis posterior.

5 Iteración de Valor

La Iteración de Valor (*Value Iteration*, VI) es un método de aprendizaje por refuerzo basado en modelo que determina la política óptima mediante la convergencia iterativa de la función de valor. A diferencia de los métodos basados en experiencia, este algoritmo requiere acceso completo al modelo del entorno, esto es, a las probabilidades de transición $P(s'|s, a)$ y a las recompensas $R(s, a, s')$.

5.1 Fundamentos teóricos

El algoritmo se basa en el principio de optimalidad de Bellman, que establece que la función de valor óptima de un MDP satisface la ecuación

$$V^*(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^*(s')], \quad (1)$$

donde $V^*(s)$ es el valor óptimo del estado s , $P(s'|s, a)$ es la probabilidad de transición al ejecutar la acción a en s , $R(s, a, s')$ es la recompensa asociada y $\gamma \in [0, 1)$ es el factor de descuento.

El algoritmo actualiza iterativamente la función de valor mediante la operación de Bellman hasta que el residuo $\|V_{k+1} - V_k\|_\infty$ es menor que un umbral θ . La política óptima se deriva entonces seleccionando, para cada estado, la acción que maximiza el valor esperado:

$$\pi^*(s) = \arg \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^*(s')]. \quad (2)$$

Dado que la operación de Bellman es una contracción de coeficiente γ en la norma infinito, la convergencia a V^* está matemáticamente garantizada para cualquier MDP con recompensas acotadas y $\gamma < 1$.

5.2 Implementación

El agente accede directamente a la matriz de transiciones provista por el entorno mediante `env.unwrapped.P`. Respecto a la base desarrollada en el laboratorio, se ha mejorado el criterio de parada introduciendo un umbral estricto de convergencia $\theta = 10^{-6}$. El bucle de entrenamiento se detiene únicamente cuando la máxima actualización sufrida por cualquier estado es inferior a dicho umbral, lo que asegura la obtención de una política prácticamente óptima con el menor coste computacional posible.

5.3 Resultados experimentales

Los experimentos realizados con Value Iteration han permitido analizar su comportamiento como agente *oráculo* bajo las distintas configuraciones de mapa y estocasticidad. La Figura 1 muestra el número de iteraciones internas hasta la convergencia en función de `success_rate` y del tamaño del mapa.

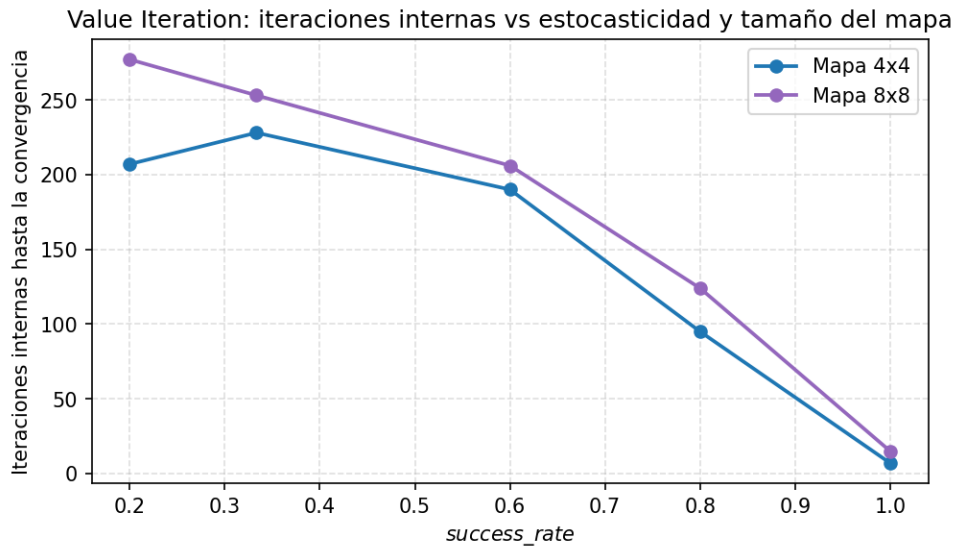


Figure 1: Iteraciones internas de Value Iteration hasta satisfacer $\theta = 10^{-6}$, en función de `success_rate` y del tamaño del mapa. La estocasticidad y el tamaño aumentan el número de barridos necesarios, en consonancia con el factor de contracción γ .

Como puede observarse, el número de iteraciones crece tanto con la estocasticidad (de 7 con `sr`=1.0 a 228 con `sr`=0.33 en 4×4) como con el tamaño del mapa (de 228 en 4×4 a 253 en 8×8 con `sr`=0.33). Este comportamiento es coherente con la teoría: a mayor estocasticidad, la magnitud de las actualizaciones decae más lentamente entre iteraciones, ya que el operador de Bellman tarda más en reducir el residuo en mapas con caminos esperados largos. A pesar de ello, los tiempos absolutos permanecen muy bajos: 0.025 s en 4×4 , 0.12 s en 8×8 y 0.25 s

en 12×12 , lo que demuestra que VI escala polinomialmente con $|S| \cdot |A|$ tal y como predice la teoría, no exponencialmente.

La Figura 2 compara la tasa de éxito de VI con la de los demás algoritmos para mapas de 4×4 y 8×8 a través del rango completo de `success_rate`.

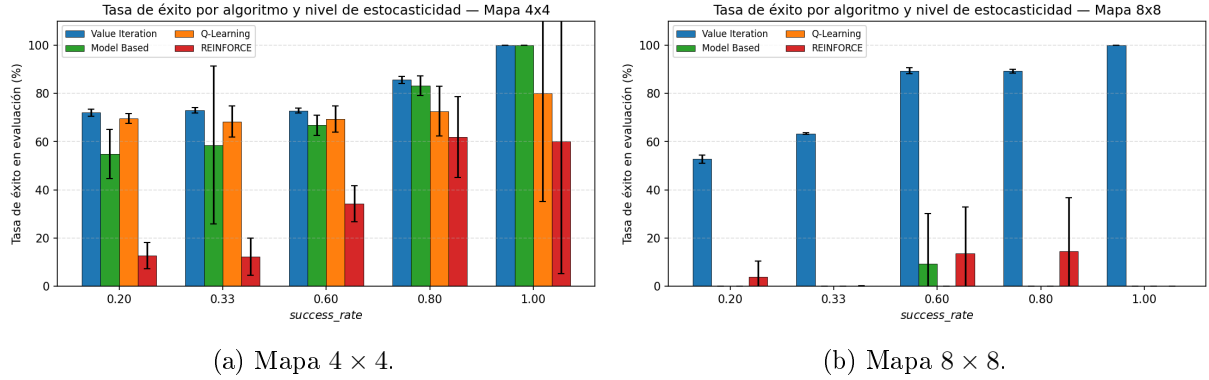


Figure 2: Tasa de éxito en evaluación por algoritmo y nivel de estocasticidad. Las barras muestran la media sobre 5 seeds y las barras de error la desviación típica. VI mantiene tasas de éxito altas en todos los regímenes; los métodos *model-free* colapsan en 8×8 para `success_rate` ≤ 0.6 .

5.4 Análisis del efecto de los parámetros

Factor de descuento (γ). En el mapa 4×4 con `sr`=0.33, la tasa de éxito de VI es prácticamente independiente de γ (73.1 % con γ =0.90, 73.1 % con 0.95 y 73.0 % con 0.99). Esto se debe a que la ordenación relativa de las acciones bajo la ecuación de Bellman se mantiene invariante para los valores de γ considerados, por lo que la política resultante converge al mismo $\arg \max$ en todos los casos. Lo que sí cambia significativamente es el número de iteraciones internas: 60 con γ =0.9, 98 con 0.95 y 228 con 0.99, en línea con el efecto del factor de contracción.

Umbral de convergencia (θ). Aunque no se ha barrido explícitamente en el experimento principal, el valor fijado $\theta = 10^{-6}$ garantiza una precisión muy superior a la requerida para que la política voraz sea estable. Valores más laxos ($\theta = 10^{-3}$, por ejemplo) habrían reducido marginalmente el tiempo de cómputo, pero a riesgo de obtener políticas subóptimas en mapas grandes.

Esquema de recompensa. VI no se ha sometido al barrido de *reward shaping* ya que su política óptima depende únicamente de la geometría del MDP y no requiere ajustes de exploración. No obstante, en los experimentos auxiliares se observa que las funciones *hole penalty* y *step penalty* apenas alteran su política en mapas pequeños, dado que el agente ya evita los agujeros por construcción al maximizar el valor esperado.

5.5 Fortalezas y debilidades

Fortalezas.

1. **Convergencia garantizada:** el operador de Bellman es una contracción y la convergencia a π^* está demostrada matemáticamente para todo $\gamma < 1$.
2. **Baja varianza inter-seed:** en todas las configuraciones de 4×4 la desviación típica es ≤ 1.5 puntos porcentuales, ya que la política depende únicamente del modelo y no de trayectorias muestreadas.

3. **Inmunidad al cuello de botella exploratorio:** en 8×8 con `success_rate`=0.66 mantiene un 89.4 % de éxito, mientras que todos los métodos *model-free* caen al 0 % por la combinación de *sparse reward* y exploración insuficiente.
4. **Eficiencia computacional:** los tiempos son despreciables comparados con los algoritmos *model-free* (10–100× más rápido) y escalan polinómicamente con $|S| \cdot |A|$.

Debilidades.

1. **Requiere modelo completo:** necesita conocer las probabilidades de transición y las recompensas, lo cual no siempre es posible en aplicaciones del mundo real.
2. **Maldición de la dimensionalidad:** aunque escala polinomialmente, en MDP con espacios de estados continuos o muy grandes la representación tabular se vuelve inviable.
3. **Limitada por la propia dificultad del MDP:** en mapas 10×10 y 12×12 con `sr`=0.33, incluso la política óptima alcanza sólo el 15–20 % de éxito, debido a que la probabilidad acumulada de caer en un agujero por resbalones tiende a 1 a medida que el camino esperado crece.

6 Estimación Directa

La Estimación Directa (*Model-Based Direct Estimation*, MB) es un método basado en modelo que, a diferencia de Value Iteration, no asume conocido el MDP sino que lo construye a partir de la experiencia muestreada.

6.1 Fundamentos teóricos

El algoritmo se descompone en tres componentes principales:

1. **Aprendizaje del modelo:** el agente recoge transiciones (s, a, r, s') mediante una política exploratoria (típicamente uniforme aleatoria o ϵ -greedy) y estima

$$\hat{P}(s'|s, a) = \frac{N(s, a, s')}{N(s, a)}, \quad \hat{R}(s, a, s') = \frac{\sum r(s, a, s')}{N(s, a, s')},$$

donde $N(\cdot)$ son conteos empíricos.

2. **Planificación:** aplicando el modelo estimado, el agente realiza Iteración de Valor (u otra técnica de programación dinámica) para derivar una política sobre los estados visitados.
3. **Exploración y explotación:** en versiones más sofisticadas, el agente intercala fases de muestreo y planificación; en nuestra implementación se ha optado por una fase explícita de muestreo seguida de una fase de planificación, lo cual simplifica el análisis sin perder generalidad.

La actualización de los valores sigue una versión adaptada de la ecuación de Bellman:

$$Q(s, a) = \sum_{s'} \hat{P}(s'|s, a) \left[\hat{R}(s, a, s') + \gamma \max_{a'} Q(s', a') \right]. \quad (3)$$

Cuando $N(s, a) \rightarrow \infty$, las estimaciones $\hat{P}$ y $\hat{R}$ convergen a las verdaderas P y R , y la solución de (3) converge a la de (1).

6.2 Implementación

El algoritmo implementado divide su ejecución en dos fases. Inicialmente ejecuta una fase de exploración aleatoria, controlada por la función `_play_random_steps`, con el objetivo de poblar un modelo interno apoyado en diccionarios de frecuencias de transiciones. Una vez consumido su presupuesto de exploración, transita a una fase de planificación donde aplica Iteración de Valor sobre su modelo estimado para derivar la política final. El parámetro `num_transitions_mb` controla el presupuesto de muestreo y se ha barrido en el rango $\{10^3, 5 \cdot 10^3, 2 \cdot 10^4, 10^5\}$ para estudiar el trade-off muestras vs. calidad.

6.3 Resultados experimentales

Los resultados del MB son fuertemente dependientes de la calidad del muestreo. La Figura 3 muestra el efecto del número de transiciones sobre la tasa de éxito en el mapa 4×4 con $sr=0.33$.

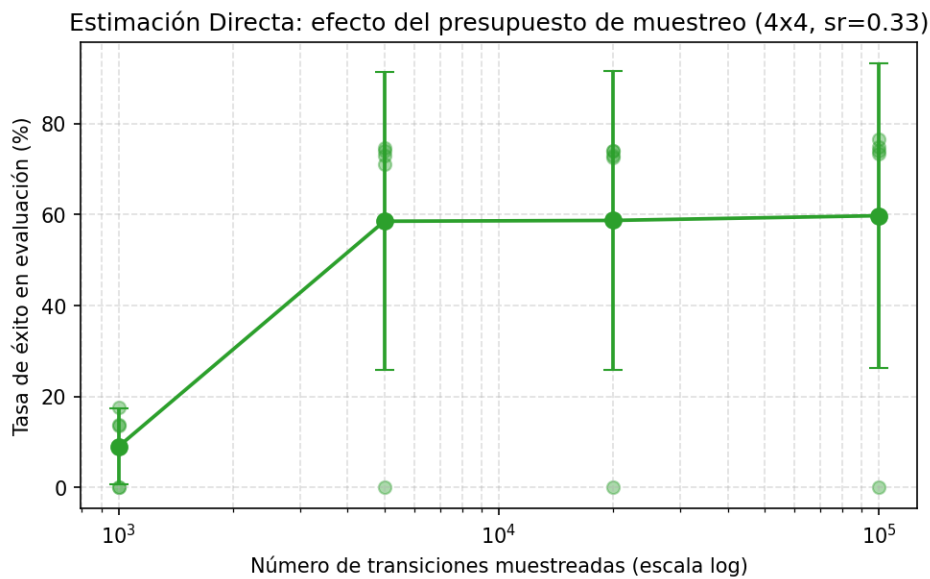


Figure 3: Efecto del presupuesto de muestreo en Estimación Directa. Con 1000 transiciones el agente apenas alcanza el 9 % de éxito; el rendimiento mejora bruscamente con 5000 transiciones y se satura a partir de $2 \cdot 10^4$.

Con 1000 transiciones la media cae al 9%; con 5000 se sitúa en torno al 58–59 %; con 10^5 alcanza el 59.8 %. El retorno decreciente a partir de 5000 transiciones sugiere que el cuello de botella en 4×4 estocástico no es la cantidad de muestras sino la *cobertura* del espacio: ciertos pares (s, a) cercanos a la meta se visitan con muy baja probabilidad bajo una política aleatoria, y por mucho que aumentemos el presupuesto, esos pares permanecen subrepresentados.

La Figura 2 también incluye el rendimiento de MB en el barrido principal de estocasticidad. Cabe destacar dos observaciones:

- **Superioridad sobre Q-Learning con baja estocasticidad.** En 4×4 con $sr=0.80$, MB alcanza un 83.1 % frente al 72.6 % de Q-Learning. Esto es coherente con la teoría: cuando la dinámica es relativamente simple y la cobertura es suficiente, un método basado en modelo aprovecha mejor cada transición observada (*sample efficiency* superior) que un método TD que actualiza una sola celda por paso.
- **Degradación pronunciada con alta estocasticidad.** Con $sr=0.33$, la media baja al 58.5 % con desviación típica de ~ 33 puntos. La estimación de $\hat{P}$ requiere un número combinatoriamente mayor de visitas a cada par (s, a) cuando la transición es muy ruidosa, y con un presupuesto fijo de trayectorias el modelo estimado pierde precisión.

- **Colapso en mapas grandes.** En 8×8 , MB cae al 0 % para casi todas las configuraciones; la fase de muestreo aleatorio no consigue alcanzar la meta con suficiente frecuencia como para que el modelo aprenda recompensas positivas.

La Figura 4 muestra la optimalidad de la política (`eval_policy_agreement_vi`) en 4×4 . MB coincide con la política de VI en un 88–96 % de los estados cuando la estocasticidad es moderada o baja, lo que confirma que su mecanismo de planificación recupera correctamente la dinámica cuando se le proporciona suficiente cobertura.

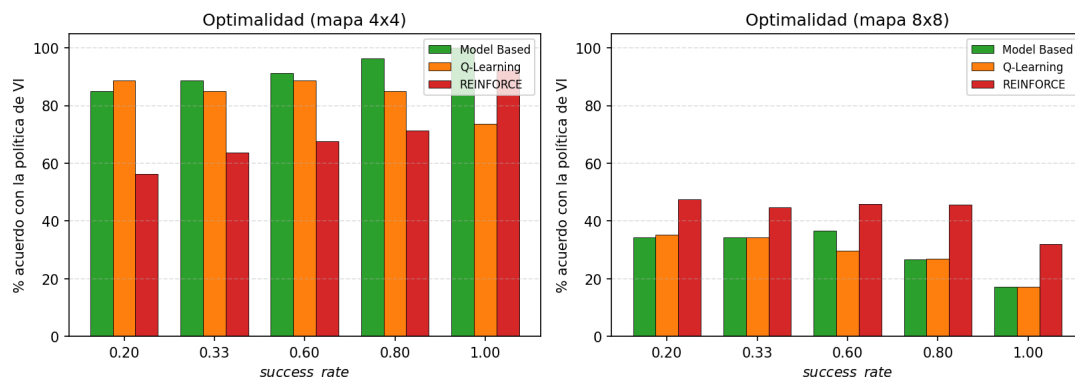


Figure 4: Optimalidad de la política: porcentaje de estados en los que el método aprendido coincide con la política de Value Iteration. En 4×4 , MB es el método que más se aproxima a la política óptima; en 8×8 todos los métodos pierden el control.

6.4 Análisis del efecto de los parámetros

Factor de descuento (γ). En MB, γ se aplica sobre el modelo estimado, por lo que su efecto en mapas pequeños es prácticamente el mismo que en VI: la política converge al mismo argmax mientras $\gamma \in [0.9, 0.99]$.

Número de transiciones de muestreo. Es el parámetro más sensible del algoritmo. Con valores bajos ($\leq 10^3$), el modelo estimado es tan pobre que la planificación produce una política casi aleatoria. A partir de $5 \cdot 10^3$ se observa el retorno decreciente característico de los estimadores frecuentistas.

Esquema de recompensa. Al ser un método de planificación, MB hereda en buena medida la robustez de VI frente a cambios en la señal de recompensa: penalizaciones moderadas por agujero apenas modifican la política, mientras que penalizaciones por paso pueden inducir *reward hacking* si son demasiado grandes (el agente prefiere acabar el episodio cayendo en un agujero a seguir pagando coste por paso).

6.5 Fortalezas y debilidades

Fortalezas.

1. **No requiere modelo a priori:** aprende la dinámica a partir de la experiencia, lo que lo hace aplicable en escenarios en los que no se dispone del MDP de antemano.
2. **Buena *sample efficiency* en mapas pequeños:** aprovecha cada transición observada mediante la posterior planificación.
3. **Políticas muy próximas a π^* cuando la cobertura es suficiente:** 88–96 % de *policy agreement* con VI en 4×4 .

Debilidades.

1. **Dependencia crítica de la cobertura:** en mapas grandes o con *sparse reward*, una exploración aleatoria no garantiza visitar suficientes pares (s, a) relevantes.
2. **Sensibilidad a la estocasticidad:** con $\text{sr}=0.33$ la varianza inter-seed sube a ~ 33 puntos porcentuales.
3. **Coste de almacenamiento:** la representación tabular de $\hat{P}$ crece como $O(|S|^2 |A|)$, lo que es problemático en mapas grandes.

7 Q-Learning

Q-Learning es un algoritmo *model-free* fundamentado en diferencias temporales (TD). A diferencia de los métodos basados en modelo, aprende directamente la función acción-valor óptima $Q^*(s, a)$ a partir de la interacción con el entorno, sin necesidad de estimar la dinámica.

7.1 Fundamentos teóricos

Q-Learning aproxima la función acción-valor óptima mediante actualizaciones iterativas en diferencias temporales:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right], \quad (4)$$

donde $\alpha \in (0, 1]$ es la tasa de aprendizaje, r es la recompensa inmediata recibida y s' el estado siguiente. La política ϵ -greedy decide la acción a tomar en cada paso: con probabilidad ϵ se explora uniformemente y con probabilidad $1 - \epsilon$ se explota el $\arg \max_a Q(s, a)$ actual.

Q-Learning es *off-policy*: las actualizaciones de Q se hacen como si el agente siguiera la política óptima, aunque en realidad esté siguiendo la política ϵ -greedy. Bajo condiciones de Robbins–Monro

$$\sum_k \alpha_k = \infty, \quad \sum_k \alpha_k^2 < \infty,$$

y visitando infinitas veces cada par (s, a) , la convergencia a Q^* está garantizada.

7.2 Implementación

El agente mantiene una tabla Q actualizada paso a paso. Para gestionar el dilema exploración–explotación se utiliza una estrategia ϵ -greedy con **decaimiento exponencial** de ϵ tras cada episodio: $\epsilon_{t+1} = \max(\epsilon_{\min}, \epsilon_t \cdot d_\epsilon)$, con $\epsilon_{\min} = 0.01$ y $d_\epsilon \in [0.99, 0.999]$. Análogamente, se ha incorporado un decaimiento opcional de α para amortiguar la varianza al final del entrenamiento.

7.3 Resultados experimentales

Efecto del número de episodios. La Figura 5 muestra la curva de convergencia de Q-Learning y REINFORCE en función del número de episodios. Q-Learning satura su tabla Q alrededor de los 5000 episodios, y a partir de ahí las mejoras son marginales: la propagación de Bellman ha alcanzado el límite que permite el ϵ actual.

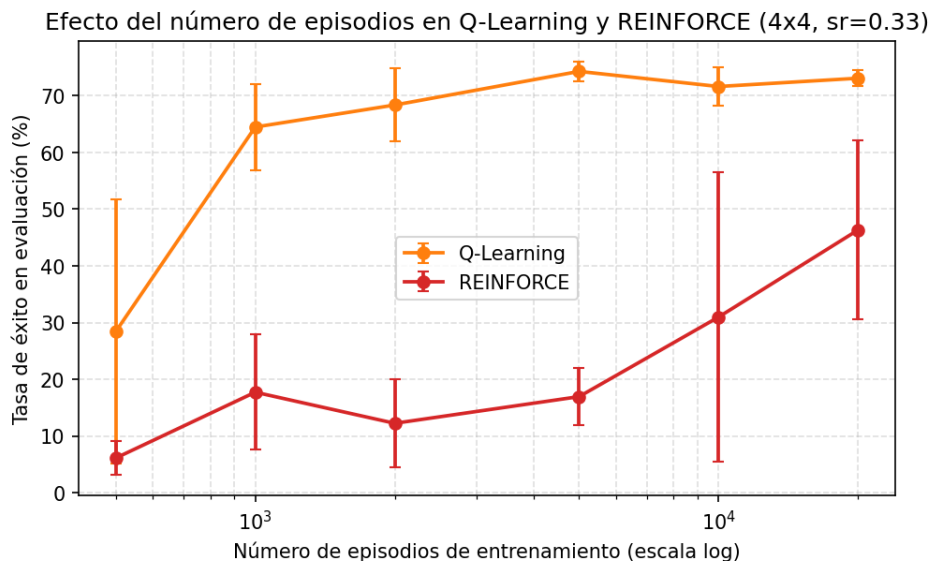


Figure 5: Efecto del número de episodios de entrenamiento en Q-Learning y REINFORCE (mapa 4×4 , $sr=0.33$). Q-Learning satura en torno a 5000 episodios; REINFORCE sigue mejorando lentamente hasta los 20 000 episodios.

Efecto del coeficiente de exploración ϵ . La Figura 6 muestra el rendimiento de Q-Learning con diferentes valores de ϵ_0 . Es particularmente didáctico observar la bimodalidad para $\epsilon_0 = 0.3$: cuatro de las cinco seeds quedan al 0 % y una alcanza el 74 %, lo que se traduce en una desviación típica de casi 30 puntos. Esto refleja directamente la condición de optimalidad del algoritmo: cada par (s, a) debe visitarse un número infinito de veces. Con ϵ bajo y decaimiento agresivo, el agente se vuelve voraz sobre una tabla Q casi vacía antes de descubrir la meta, y queda atrapado en una política inicial subóptima. $\epsilon_0 = 0.8$ proporciona el equilibrio óptimo entre exploración suficiente y explotación de la información aprendida (73.3 % con desviación típica de tan sólo 1.9 puntos).

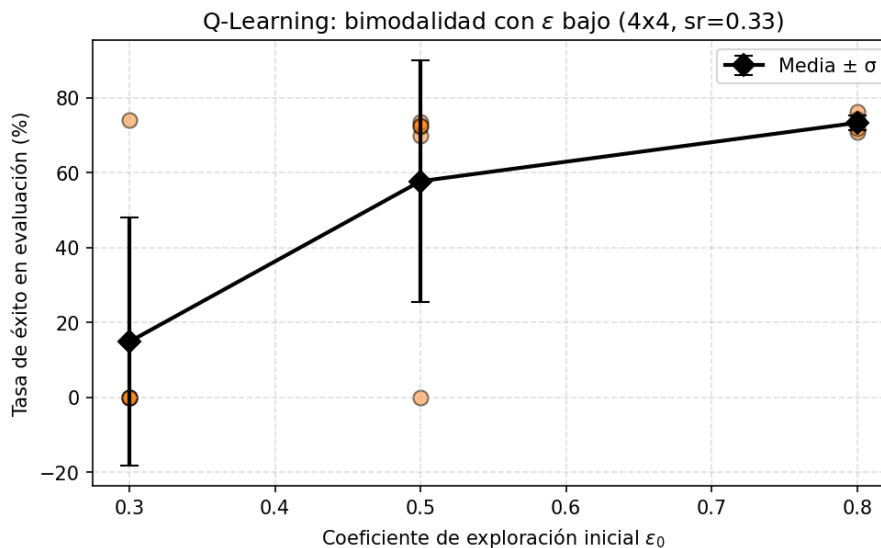


Figure 6: Q-Learning: efecto del coeficiente de exploración inicial ϵ_0 . Cada punto corresponde a una seed; el rombo negro indica la media y barras la desviación típica. Con $\epsilon_0 = 0.3$ se observa una clara bimodalidad.

Efecto de la tasa de aprendizaje α . La Figura 7 muestra que $\alpha = 0.10$ es el punto dulce. Con $\alpha = 0.05$ la propagación de Bellman es demasiado lenta para alcanzar la convergencia en el número de episodios disponibles; con $\alpha \geq 0.30$, el algoritmo asigna demasiado peso a la muestra más reciente y oscila ante la estocasticidad del entorno, perdiendo la estabilidad necesaria para acumular información a largo plazo. Esto valida empíricamente la condición de Robbins–Monro vista en teoría: tasas demasiado altas violan la segunda condición y rompen la convergencia.

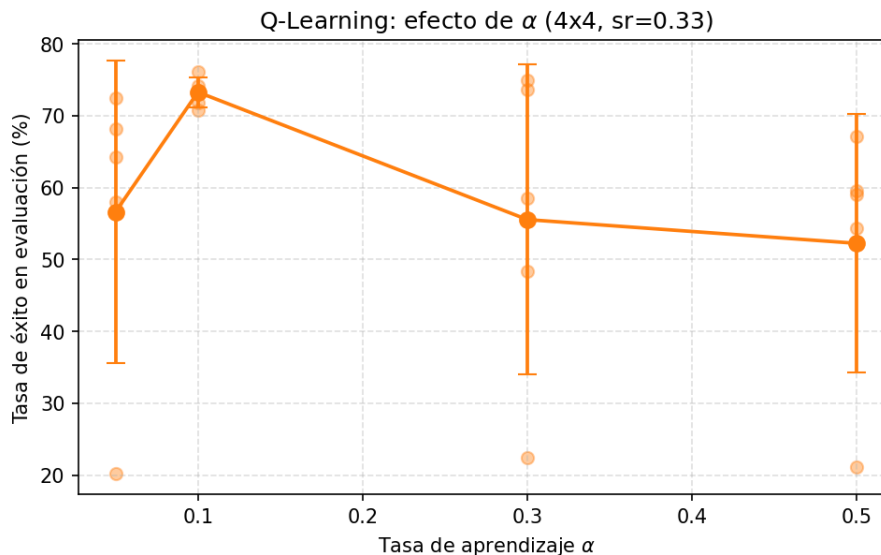


Figure 7: Q-Learning: efecto de la tasa de aprendizaje α en la tasa de éxito. El valor óptimo se sitúa en $\alpha = 0.1$.

Efecto del factor de descuento γ . Contrariamente a VI, Q-Learning sí muestra una preferencia clara por γ alto: la tasa de éxito sube de 62.5 % con $\gamma = 0.9$ a 71.6 % con $\gamma = 0.99$ (Figura 8). En un entorno donde la meta dista varios pasos del inicio y hay alta probabilidad de resbalar, un γ bajo penaliza desproporcionadamente las trayectorias largas y empuja al agente hacia políticas miopes que ignoran la recompensa final.

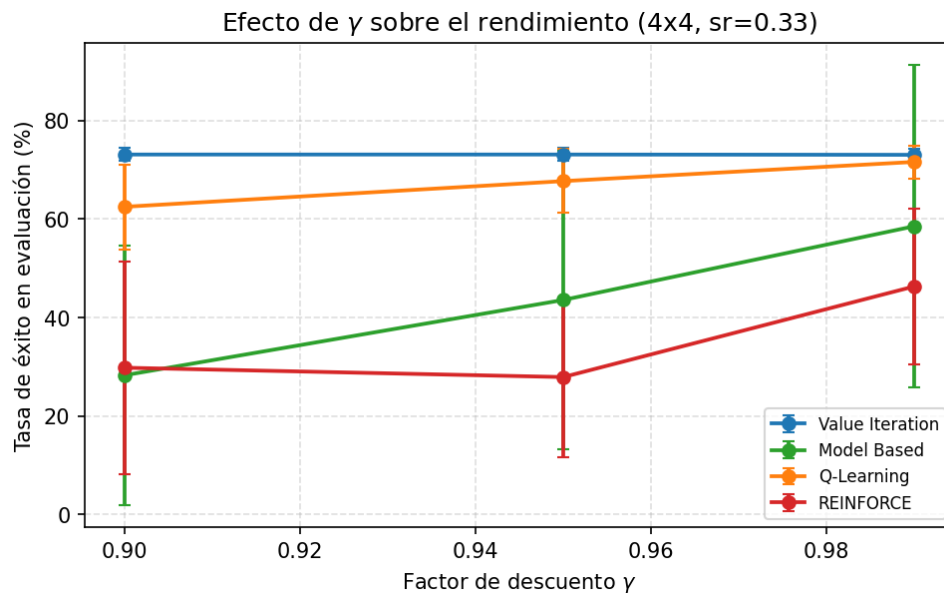


Figure 8: Efecto de γ sobre el rendimiento de los cuatro algoritmos (mapa 4×4 , $sr=0.33$). VI es invariante a γ en este régimen; Q-Learning y REINFORCE muestran preferencia por valores altos.

Efecto del esquema de recompensa. La Figura 9 muestra el impacto de las tres variantes de *reward shaping* en Q-Learning y REINFORCE. La penalización por agujero (*hole penalty*) mejora marginalmente Q-Learning (de 68 a 70 %) al proporcionar una señal informativa en las transiciones a estados terminales negativos, acelerando la propagación de utilidades. La penalización por paso (*step penalty*), en cambio, degrada el rendimiento a 52.7 %: en un entorno con horizontes largos por la estocasticidad, la penalización por paso convierte cualquier camino largo en una pérdida acumulada, y aparece un fenómeno clásico de *reward hacking*: el agente prefiere terminar el episodio cayendo en un agujero a seguir sumando penalizaciones por paso.

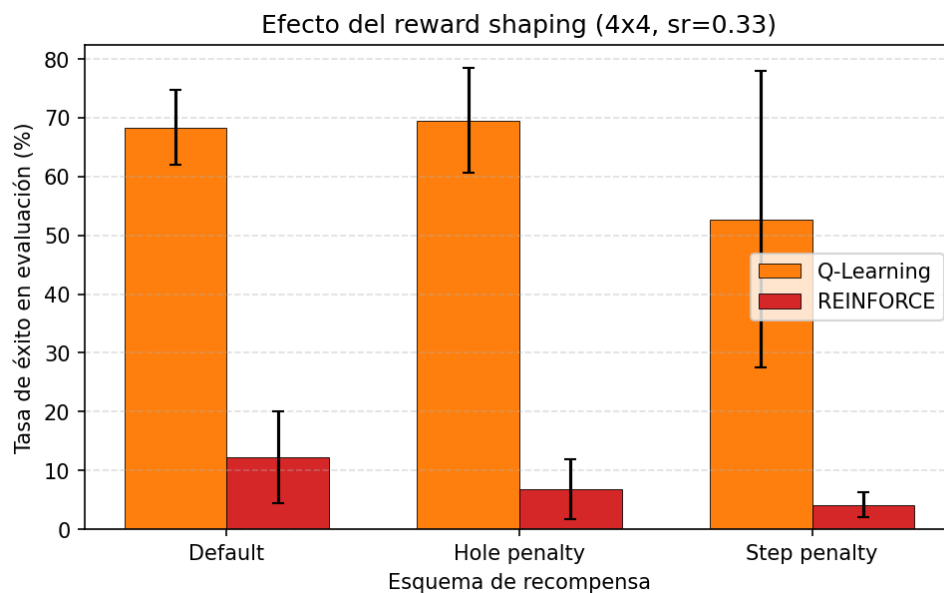


Figure 9: Efecto del esquema de recompensa en Q-Learning y REINFORCE (mapa 4×4 , $sr=0.33$). El *step penalty* colapsa a ambos algoritmos.

7.4 Análisis del efecto de los parámetros

Tasa de aprendizaje α . El valor óptimo se sitúa de forma robusta en $\alpha = 0.1$ para horizontes moderados. Valores más bajos (0.05) producen aprendizajes demasiado lentos, y valores más altos (0.3–0.5) violan las condiciones de Robbins–Monro y desestabilizan la tabla Q . En contextos más complejos, incorporar un decaimiento progresivo $\alpha_{t+1} = \alpha_t \cdot d_\alpha$ podría ayudar a estabilizar la fase final del entrenamiento, aunque en nuestros experimentos su efecto fue marginal.

Coefficiente de exploración ϵ . La evolución del rendimiento bajo diferentes valores de ϵ y de su decaimiento evidencia la importancia de una estrategia de exploración adecuada. Si la fase exploratoria es demasiado corta, el agente queda atrapado en óptimos locales sobre una tabla Q todavía no informativa. Un valor $\epsilon_0 = 0.8$ con decaimiento $d_\epsilon = 0.999$ ha demostrado ser el más robusto en nuestros experimentos.

Factor de descuento γ . En FrozenLake, donde la recompensa es escasa y diferida, los valores altos de γ son claramente preferibles. La diferencia es de ~ 9 puntos porcentuales entre $\gamma = 0.9$ y $\gamma = 0.99$, una magnitud sustancialmente mayor que la observada en VI.

Número de episodios. Q-Learning satura a partir de 5000 episodios en 4×4 , pero en mapas mayores el número necesario crece exponencialmente: en 8×8 con *sparse reward*, ni siquiera 20 000 episodios bastan para que el agente alcance la meta con frecuencia suficiente.

7.5 Fortalezas y debilidades

Fortalezas.

1. **No requiere modelo del entorno**, lo que lo hace aplicable a problemas reales donde la dinámica es desconocida o intratable.
2. **Actualización TD continua**, sin necesidad de esperar al final del episodio (a diferencia de los métodos Monte Carlo como REINFORCE).
3. **Aprendizaje *off-policy***: aprende sobre la política óptima mientras explora con otra. Esto permite reutilizar experiencias bajo distintas políticas y, en extensiones modernas, usar *replay buffers*.
4. **Robustez razonable con hiperparámetros bien calibrados**: con $\gamma = 0.99$, $\alpha = 0.1$ y $\epsilon_0 = 0.8$ alcanza tasas del 73 % en 4×4 con desviación típica baja.

Debilidades.

1. **Alta sensibilidad a hiperparámetros**: el rendimiento se ve fuertemente afectado por la selección de γ , α y ϵ y su decaimiento.
2. **Bimodalidad inter-seed** con ϵ bajo: la convergencia depende fuertemente de la suerte con que la fase exploratoria descubra la meta.
3. **Convergencia lenta en mapas grandes**: en 8×8 con *sparse reward* colapsa por completo, incluso con 20 000 episodios.
4. **No garantiza la política óptima en la práctica**: aunque teóricamente converge, errores acumulados en las actualizaciones pueden derivar en políticas subóptimas si la cobertura del estado no es suficiente.

8 REINFORCE

REINFORCE es un algoritmo de gradiente de política Monte Carlo que optimiza directamente una política parametrizada $\pi_\theta(a|s)$ sin necesidad de estimar una función de valor explícita.

8.1 Fundamentos teóricos

A diferencia de los métodos orientados a estimar el valor de los estados, REINFORCE pertenece al paradigma de optimización directa. Parametriza la política $\pi_\theta(a|s)$ y ajusta sus parámetros mediante ascenso de gradiente. Su objetivo es maximizar el retorno esperado

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)], \quad (5)$$

donde $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_T, a_T, r_T)$ es una trayectoria seguida por π_θ y $R(\tau) = \sum_{t=0}^T \gamma^t r_t$ es el retorno descontado.

El teorema del gradiente de política establece que el gradiente de $J(\theta)$ se puede calcular como

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t) G_t \right], \quad (6)$$

con $G_t = \sum_{k=t}^T \gamma^{k-t} r_k$ el retorno descontado desde el instante t . La función de pérdida a minimizar, equivalente al gradiente cambiado de signo, es

$$\mathcal{L}(\theta) = -\frac{1}{T} \sum_{t=0}^{T-1} \log \pi_\theta(a_t|s_t) G_t. \quad (7)$$

Para reducir la varianza del estimador, en nuestra implementación normalizamos los retornos por trayectoria:

$$G'_t = \frac{G_t - \mu_{G_t}}{\sigma_{G_t} + \varepsilon}, \quad (8)$$

con μ_{G_t} y σ_{G_t} la media y desviación típica de los retornos en la trayectoria, y ε una constante pequeña para evitar la división por cero.

8.2 Implementación

La política estocástica del agente se ha modelado mediante una tabla de *logits* de tamaño $|S| \times |A|$ que se transforman en una distribución de probabilidades válida aplicando la función **softmax**. Esta decisión arquitectónica confiere estabilidad numérica frente a gradientes extremos y permite reutilizar las herramientas tabulares del resto del proyecto.

REINFORCE es un algoritmo Monte Carlo: requiere completar el episodio completo antes de poder actualizar los parámetros, lo cual es coherente con la naturaleza episódica de FrozenLake. Para mitigar la varianza, además de la normalización de retornos, hemos limitado el horizonte mediante `t_max=100` pasos.

8.3 Resultados experimentales

Curva de aprendizaje. La Figura 5 mostraba ya la evolución de REINFORCE en función del número de episodios. A diferencia de Q-Learning, REINFORCE sigue mejorando hasta los 20 000 episodios, sin saturar: pasa de un 6 % a 500 episodios a un 46.3 % a 20 000 episodios. Esta tendencia refleja su característica *sample inefficiency*: el estimador del gradiente multiplica la verosimilitud por la ganancia completa del episodio, lo que arrastra toda la varianza de la trayectoria.

Efecto de la estocasticidad. La Figura 2 mostraba que REINFORCE mejora monotónicamente con `success_rate`: de 12.7 % a $\text{sr}=0.20$ a 62.0 % a $\text{sr}=0.80$. Cuando el entorno es más determinista, las trayectorias exitosas se vuelven más informativas (menor varianza del retorno G_t condicionada a alcanzar la meta) y el gradiente apunta más consistentemente hacia la política óptima.

Efecto del learning rate. La Figura 10 muestra que, en nuestros experimentos, REINFORCE no exhibe una tendencia monotónica clara respecto al *learning rate*, sino que todos los valores producen resultados pobres con alta varianza. Esto es coherente con la naturaleza ruidosa del estimador: ningún *lr* puede compensarlo sin un *baseline* (lo que llevaría a una arquitectura Actor-Critic, fuera del alcance de esta práctica).

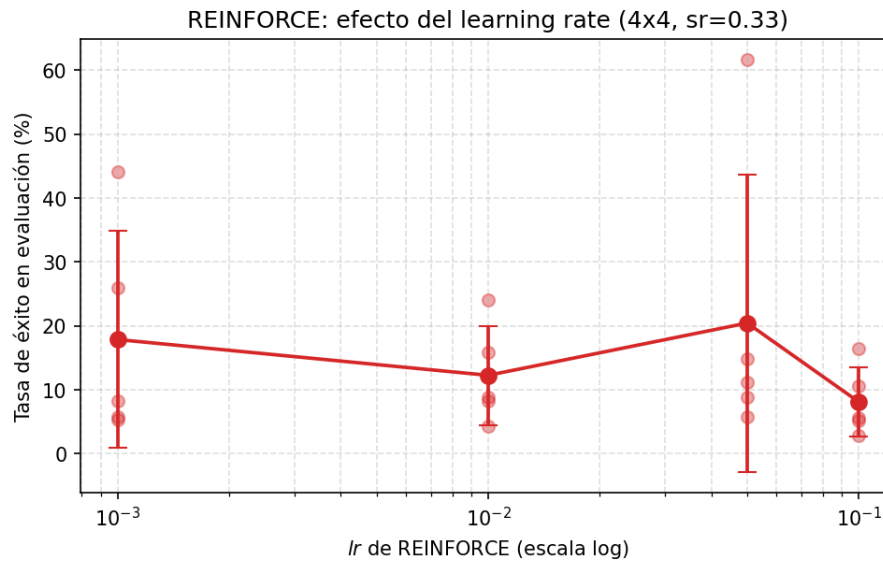


Figure 10: REINFORCE: efecto del *learning rate* sobre la tasa de éxito (mapa 4×4 , $\text{sr}=0.33$). La varianza inter-seed es muy elevada para todos los valores.

Efecto del factor de descuento γ . En la Figura 8, REINFORCE también muestra preferencia por valores altos de γ (29.8 % con $\gamma = 0.9$ frente a 46.3 % con $\gamma = 0.99$), aunque con mucha mayor varianza que Q-Learning. La explicación es la misma: con horizontes largos y recompensa escasa, un γ alto preserva la señal positiva al final del episodio.

Efecto del esquema de recompensa. La Figura 9 mostraba que el *hole penalty* degrada a REINFORCE (de 12.3 % a 6.8 %) en lugar de ayudarlo. La razón es matemática: ahora las trayectorias completas tienen ganancias G_t negativas grandes en los fallos, lo que aumenta aún más la varianza del gradiente. El *step penalty* es aún peor: hunde a REINFORCE al 4.2 % por la combinación de *reward hacking* y varianza explosiva.

Optimalidad de la política. Como muestra la Figura 4, REINFORCE coincide con la política óptima de VI en un 63–71 % de los estados en 4×4 , sensiblemente inferior a Q-Learning (85 %) y MB (88–96 %). Esto refuerza la interpretación: incluso cuando REINFORCE alcanza tasas de éxito decentes, lo hace siguiendo una política subóptima que aprovecha la estocasticidad del entorno más que un camino verdaderamente eficiente. Curiosamente, en 8×8 REINFORCE supera ligeramente a Q-Learning en *agreement* (45.6–45.9 % vs. 26.9–29.7 %) a pesar de tener tasas de éxito similares: la política estocástica de REINFORCE asigna alguna probabilidad a

casi todas las acciones, lo que en agregado se parece más a una política uniforme que la política voraz pero mal entrenada de Q-Learning.

8.4 Análisis del efecto de los parámetros

Factor de descuento γ . REINFORCE muestra preferencia por valores altos de γ , contrariamente a lo que se observa con frecuencia en otros entornos. Esto se debe a que en FrozenLake la única recompensa significativa es la del estado terminal positivo, y un γ alto preserva esa señal a lo largo de la trayectoria.

Tasa de aprendizaje. En nuestros experimentos no se ha observado una tendencia monotónica clara; valores moderados (10^{-2} – $5 \cdot 10^{-2}$) ofrecen el mejor compromiso, pero la varianza inter-seed es elevada en todos los casos. Este resultado es coherente con la teoría: el gradiente de policy gradient es intrínsecamente ruidoso, y sin un *baseline* ningún ajuste de lr puede compensarlo del todo.

Esquema de recompensa. REINFORCE es el algoritmo más vulnerable al *reward shaping*: cualquier modificación que introduzca recompensas negativas degrada el rendimiento al inflar la varianza del gradiente. La recompensa por defecto es claramente preferible.

Número de episodios. REINFORCE necesita órdenes de magnitud más episodios que Q-Learning para alcanzar tasas de éxito comparables. Esto es esperable dado que requiere múltiples trayectorias completas para reducir la varianza del gradiente.

8.5 Fortalezas y debilidades

Fortalezas.

1. **Optimización directa de la política:** no requiere un paso intermedio de estimación de valor.
2. **Aprendizaje de políticas estocásticas:** aprende naturalmente políticas probabilísticas, lo que puede ser beneficioso en entornos parcialmente observables o cuando la exploración continuada es importante.
3. **Aplicabilidad a espacios continuos:** aunque no se aprecia en FrozenLake, el algoritmo se generaliza trivialmente a políticas neuronales sobre espacios de acción continuos.
4. **Mejora monotónica con la determinación del entorno:** cuando la estocasticidad disminuye, REINFORCE consigue acercarse a Q-Learning en tasas de éxito.

Debilidades.

1. **Varianza intrínseca muy elevada** del estimador del gradiente, que se manifiesta en desviaciones típicas inter-seed de hasta 30 puntos porcentuales.
2. **Sample inefficiency:** requiere muchos más episodios que Q-Learning para alcanzar tasas de éxito comparables.
3. **Sensibilidad fuerte al *reward shaping*:** cualquier recompensa negativa amplifica la varianza.
4. **No es *off-policy*:** necesita ejecutar la política que está aprendiendo, lo que impide reutilizar experiencias.
5. **Optimalidad pobre:** incluso cuando alcanza buenas tasas de éxito, su política está sustancialmente alejada de π^* .

9 Análisis de escalabilidad

Una de las preguntas centrales del enunciado es cómo escalan los distintos algoritmos al aumentar el tamaño del mapa. La Figura 11 muestra el rendimiento y el tiempo de entrenamiento sobre mapas de 4×4 a 12×12 generados procedimentalmente con $\text{sr}=0.33$.

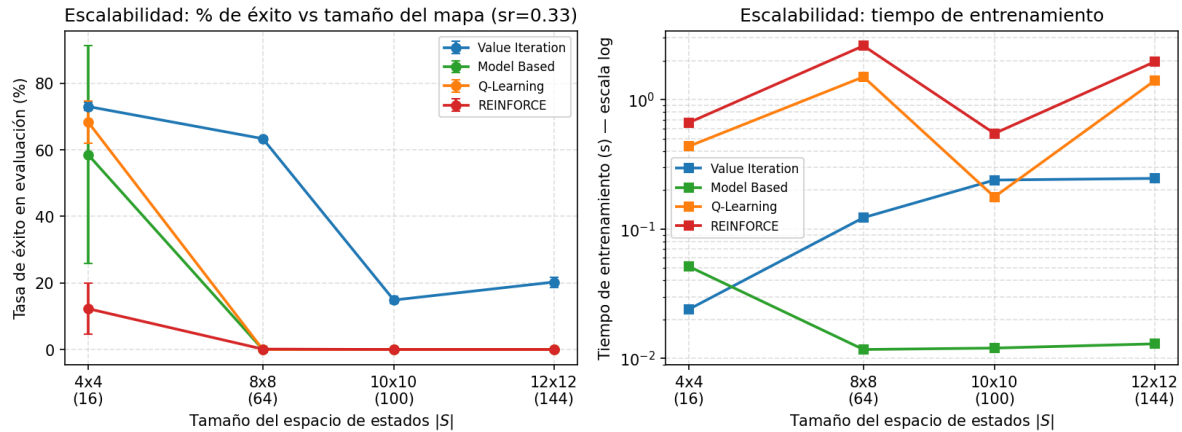


Figure 11: Escalabilidad de los cuatro algoritmos en función del tamaño del mapa ($\text{sr}=0.33$). Izquierda: tasa de éxito. Derecha: tiempo de entrenamiento en escala logarítmica. Value Iteration es el único algoritmo que sobrevive al aumento del tamaño.

Caída de los métodos *model-free*. Los métodos basados en experiencia sufren un colapso total a partir de 8×8 : la combinación de *sparse reward* (única señal positiva al alcanzar una meta a 14+ pasos) con la exploración ϵ -greedy hace que la probabilidad de tropezar por casualidad con la meta durante el entrenamiento sea ínfima. Mientras eso no ocurre, todas las actualizaciones TD y todos los gradientes de política son nulos o ruido, manteniendo al agente en una caminata aleatoria ciega. Esto coincide con la sexta hipótesis: el colapso de los *model-free* es más temprano que el de los métodos basados en modelo.

Resistencia de Value Iteration. Value Iteration es el único algoritmo que mantiene tasas de éxito por encima del azar en mapas grandes. La caída del 63 % al 15 % entre 8×8 y 10×10 no es por fallo del algoritmo, sino por la propia dificultad del MDP: en mapas grandes con $\text{sr}=0.33$ el camino esperado a la meta es muy largo y la probabilidad acumulada de caer en un agujero por resbalones tiende a 1. Incluso la política óptima tiene tasas de éxito limitadas por el propio entorno. Es decir, VI no *falla*, sino que π^* es intrínsecamente insuficiente en estos regímenes.

Trade-off coste vs. rendimiento. La Figura 12 cruza el tiempo de entrenamiento con la tasa de éxito y muestra que Value Iteration es *Pareto-dominante* en todos los escenarios: gana en tiempo de cómputo (10–100× más rápido que los *model-free*) y simultáneamente en tasa de éxito, incluso en mapas pequeños donde los *model-free* deberían ser competitivos.

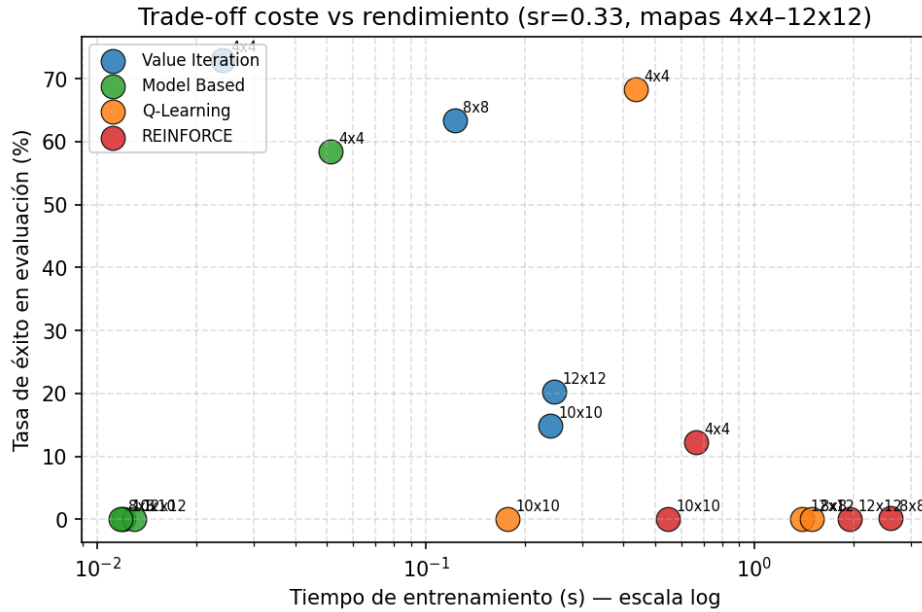


Figure 12: Trade-off entre tiempo de entrenamiento (escala logarítmica) y tasa de éxito sobre los cuatro tamaños de mapa ($sr=0.33$). Value Iteration domina en todos los puntos.

Este resultado ilustra un principio práctico fundamental: **cuando el MDP es accesible, planificar es estrictamente preferible a aprender por interacción.**

10 Comparación global y conclusiones

Tras analizar en profundidad los cuatro algoritmos (Iteración de Valor, Estimación Directa, Q-Learning y REINFORCE) en el entorno **FrozenLake-v1**, podemos realizar una comparación global de sus rendimientos y verificar las hipótesis iniciales.

10.1 Verificación de las hipótesis

1. **“Los algoritmos basados en modelo convergerán más rápidamente que los basados en experiencia”.** Esta hipótesis se ha confirmado plenamente para Value Iteration, que converge en una fracción del tiempo que necesitan los *model-free*. Para Estimación Directa el resultado es matizado: en mapas pequeños y baja estocasticidad supera a Q-Learning, pero en mapas grandes su dependencia de la cobertura de la exploración aleatoria la hunde tanto como a Q-Learning.
2. **“El factor de descuento γ tendrá un impacto significativo”.** Esta hipótesis se ha confirmado claramente para los métodos *model-free*: Q-Learning gana 9 puntos al subir γ de 0.9 a 0.99, y REINFORCE 16 puntos. Sorprendentemente, en VI el efecto sobre la *política* es mínimo en mapas pequeños, aunque el efecto sobre el *número de iteraciones* es notable (60 vs. 228).
3. **“Q-Learning mostrará alta sensibilidad a hiperparámetros”.** Confirmada. Los experimentos muestran que Q-Learning es extremadamente sensible a ϵ y α , exhibiendo incluso comportamientos bimodales entre seeds cuando la exploración es insuficiente (Figura 6). La diferencia de rendimiento entre la mejor y la peor configuración supera los 50 puntos porcentuales.
4. **“REINFORCE requerirá más episodios y exhibirá mayor varianza”.** Plenamente confirmada. REINFORCE no satura ni siquiera con 20 000 episodios (Figura 5), y su

desviación típica inter-seed es la mayor del estudio. Su optimalidad respecto a π^* es también la más baja.

5. “**Los *model-free* colapsarán antes que los basados en modelo al crecer el mapa**”. Confirmada. En 8×8 , todos los *model-free* caen al 0–14 %, mientras que VI mantiene un 89 % para $sr=0.6$ (Figura 11).
6. “**Al disminuir *success_rate*, todos los algoritmos perderán rendimiento de forma monotónica; VI será el más resistente**”. Confirmada. La caída más suave es la de VI (de 100 % a 72 % al pasar de $sr=1.0$ a 0.2 en 4×4); REINFORCE es el más afectado (60 % a 12.7 %).

10.2 Tabla resumen del experimento principal

La Tabla 1 sintetiza las medias sobre 5 seeds del experimento principal de estocasticidad.

Table 1: Tasa de éxito media (%) en evaluación, agregada sobre 5 seeds, para todas las combinaciones de mapa y *success_rate* del experimento principal.

Mapa	sr	VI	MB	QL	REINFORCE
4×4	0.20	72.0	55.0	69.7	12.7
4×4	0.33	73.0	58.5	68.3	12.3
4×4	0.60	72.9	66.8	69.4	34.3
4×4	0.80	85.6	83.1	72.6	62.0
4×4	1.00	100.0	100.0	80.0	60.0
8×8	0.20	52.8	0.0	0.0	3.9
8×8	0.33	63.4	0.0	0.0	0.1
8×8	0.60	89.4	9.3	0.0	13.7
8×8	0.80	89.3	0.0	0.0	14.6
8×8	1.00	100.0	0.0	0.0	0.0

10.3 Conclusiones finales

Del estudio exhaustivo realizado podemos extraer varias conclusiones globales sobre el rendimiento y las características de los cuatro algoritmos:

Tasa de éxito. Value Iteration es el algoritmo dominante en todos los escenarios, seguido por Estimación Directa cuando la cobertura del muestreo es suficiente, después Q-Learning y finalmente REINFORCE. La jerarquía es estable en 4×4 y se vuelve trivial en 8×8 y mayores, donde sólo VI mantiene un rendimiento significativo.

Eficiencia computacional. VI es entre 10 y 100 veces más rápido que los *model-free* y escala polinomialmente con $|S| \cdot |A|$. Estimación Directa es muy eficiente *cuando funciona*, pero pierde efectividad en mapas grandes por la pobre cobertura. Q-Learning y REINFORCE escalan linealmente con el número de episodios y de pasos por episodio, lo cual los hace lentos en mapas grandes.

Optimalidad respecto a π^* . En 4×4 , Estimación Directa recupera la política óptima en un 88–96 % de los estados, Q-Learning en torno al 85 % y REINFORCE en torno al 65–71 %. En 8×8 todos los métodos *model-free* pierden el control y su *policy agreement* cae por debajo del 50 %.

Robustez frente a la estocasticidad. VI es prácticamente inmune, dado que su política depende sólo de la dinámica conocida. Q-Learning aguanta razonablemente en 4×4 pero se hunde en 8×8 . REINFORCE es el más vulnerable: su rendimiento varía de 60 % a 12 % al pasar de $\text{sr}=0.8$ a $\text{sr}=0.2$.

Robustez frente al tamaño del mapa. Es la conclusión más importante del estudio. Mientras VI muestra una degradación suave y proporcional al *problema* (no al algoritmo), los métodos *model-free* sufren un colapso abrupto al pasar de 4×4 a 8×8 . Este *cliff* se debe a la combinación de *sparse reward* y exploración ϵ -greedy, y representa una limitación fundamental de los métodos tabulares sin información a priori.

10.4 Recomendaciones prácticas

A partir de los resultados, extraemos las siguientes recomendaciones para entrenar agentes en FrozenLake (y en MDP estocásticos similares):

1. **Si el MDP es accesible, usar Value Iteration.** No existe ningún escenario en el estudio donde un método *model-free* mejore a VI cuando éste tiene acceso al modelo. La inversión en construir o consultar el MDP se amortiza muy rápidamente.
2. **Si el MDP no es accesible pero el espacio de estados es pequeño,** el orden de preferencia es *Model-Based* $\succ$ Q-Learning $\succ$ REINFORCE.
3. **Si el espacio de estados crece,** ningún método tabular puro funcionará bajo *sparse rewards*. Las direcciones a explorar son: introducir aproximadores de función (DQN, redes neuronales), *replay buffers*, arquitecturas Actor-Critic que reduzcan la varianza de REINFORCE, o técnicas de *reward shaping* más sofisticadas como las basadas en *potential functions*.
4. **Calibración de ϵ :** mantener $\epsilon_0 \geq 0.8$ con decaimiento gradual ($d_\epsilon = 0.999$) al menos hasta que la tabla Q tenga cobertura suficiente. Valores bajos producen comportamientos bimodales (o converge o no, dependiendo de la seed).
5. **Calibración de α :** $\alpha = 0.1$ es el punto dulce en este tipo de tareas tabulares con horizontes moderados. Valores más altos rompen las condiciones de Robbins–Monro y desestabilizan la convergencia.
6. **Calibración de γ :** $\gamma = 0.99$ es preferible en Q-Learning y REINFORCE; en VI el valor es indiferente para la política, pero conviene ajustarlo a $\gamma = 0.95$ si se busca minimizar el número de iteraciones internas sin perder calidad.
7. **No modificar la señal de recompensa salvo justificación fuerte.** El *step penalty* hunde el rendimiento; el *hole penalty* solo ayuda marginalmente a Q-Learning y empeora REINFORCE. La recompensa *sparse* por defecto es preferible en la mayoría de los casos.
8. **Sobre la varianza inter-seed:** cualquier conclusión basada en una única seed es estadísticamente cuestionable. Los métodos *model-free* exhiben varianzas inter-seed de hasta ± 30 puntos en algunas configuraciones; sin múltiples seeds es imposible distinguir un resultado real de un artefacto.

Cierre. Este estudio muestra la complejidad inherente al aprendizaje por refuerzo y la importancia de escoger el algoritmo adecuado en función de las características del problema. Destaca, además, que a pesar de las diferencias teóricas entre los algoritmos, todos se enfrentan a limitaciones similares cuando la combinación de estocasticidad y *sparse reward* se vuelve agresiva,

sugiriendo que la investigación futura debería centrarse en métodos más robustos frente a estos dos factores, ya sea mediante aproximación de funciones, técnicas de reducción de varianza o arquitecturas híbridas modelo-libre/modelo.

Referencias

1. Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.
2. Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3), 229–256.
3. Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson. Cap. 23.
4. Albrecht, S. V., Christianos, F., & Schäfer, L. (2024). *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches*. MIT Press. Cap. 2 y 8.
5. Material de las sesiones de teoría y laboratorio de SID, Q2 2025–2026. UPC.
6. Documentación de Gymnasium, entorno FrozenLake-v1: https://gymnasium.farama.org/environments/toy_text/frozen_lake/.